


# Proiectarea cu Microprocesoare
# Curs 3

Andrei Voinescu

POLITEHNICA University of Bucharest

13 martie 2019

- Ce sunt întreruperile?
  - Secvențe de cod care se execută asicron cu restul programului, ca urmare a unui stimul
- Cum apar ele?
  - Factori interni (timere, terminarea unor operații lungi)
  - Factori externi (serială, butoane)
- Cum se tratează?
  - Rutine de tratare a întreruperii - RTI (en. ISR)

- Cum descriu complet un program aflat în funcționare?
  - Program Counter (PR)
  - Link Register (LR)
  - Status Register (flag de carry, flag de zero - SREG)
  - Stiva curentă
  - Registre + memorie
- Ce face procesorul pentru a trata întreruperea?
  - Salvez **contextul**
  - Găsesc adresa rutinei de tratare
  - Execut rutina de tratare
  - Refac **contextul**

- Ce trebuie să fac?
- Activez toate întreruperile
  - Întreruperile pot fi dezactivate complet cu flag-ul **I** din `SREG` - status register
- Activez întreruperea în cauză - fiecare are propriul pin de enable
- Scriu funcția de ISR

```
ISR(USART0_RX_vect)
{
    ...
}
```

**EIMSK – External Interrupt Mask Register**

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | |
|---|---|---|---|---|---|---|---|---|---|
| 0x1D (0x3D) | – | – | – | – | – | INT2 | INT1 | IINT0 | EIMSK |
| Read/Write | R | R | R | R | R | R/W | R/W | R/W | |
| Initial Value | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |

- **Bits 2:0 – INT2:0: External Interrupt Request 2 - 0 Enable**

When an INT2:0 bit is written to one and the I-bit in the Status Register (SREG) is set (one), the corresponding external pin interrupt is enabled. The Interrupt Sense Control bits in the External Interrupt Control Register, EICRA, defines whether the external interrupt is activated on rising or falling edge or level sensed. Activity on any of these pins will trigger an interrupt request even if the pin is enabled as an output. This provides a way of generating a software interrupt.

```
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>

ISR(INT0_vect)
{
    PORTC ^= (1 << PC4);
}

int main()
{
    DDRC = (1 << PC4);

    EIMSK |= (1 << INT0);

    sei();

    while (1);
}
```

- Enable la întreruperea pe care o dorim
- Enable la toate întreruperile `sei()`
- rutina de tratarea ISR
- Am păstrat bucla infinită din main!




# Cum arată codul?

```
00000000 <__vectors>:
   0:   jmp     0x7c    ; 0x7c <__ctors_end>
   4:   jmp     0x94    ; 0x94 <__vector_1>
   8:   jmp     0x90    ; 0x90 <__bad_interrupt>
   c:   jmp     0x90    ; 0x90 <__bad_interrupt>
                        ...

00000094 <__vector_1>:
  94:   push    r1
  96:   push    r0
  98:   in      r0, 0x3f        ; 63
  9a:   push    r0
  9c:   eor     r1, r1
  9e:   push    r24
  a0:   push    r25
  a2:   in      r24, 0x08       ; 8
  a4:   ldi     r25, 0x10       ; 16
  a6:   eor     r24, r25
  a8:   out     0x08, r24       ; 8
  aa:   pop     r25
  ac:   pop     r24
  ae:   pop     r0
  b0:   out     0x3f, r0        ; 63
  b2:   pop     r0
  b4:   pop     r1
  b6:   reti
```

- Tabela de vectori de întrerupere este la început
- Orice intrare nedefinită din tabelă duce la un software restart
- Ce ar fi fost diferit dacă era altă întrerupere (nu `INT0`)?




# De ce avem bucla infinită fără cod?

```
00000096 <main>:
  96:   ldi     r24, 0x10       ; 16
  98:   out     0x07, r24       ; 7
  9a:   sbi     0x1d, 0 ; 29
  9c:   sei
  9e:   rjmp    .-2
; 0x9e <main+0x8>

000000a0 <_exit>:
  a0:   cli

000000a2 <__stop_program>:
  a2:   rjmp    .-2
; 0xa2 <__stop_program>
```

- `cli` dezactivează întreruperile
- `rjmp .-2` este un loop pe loc, cu întreruperile dezactivate

```
ISR(INT0_vect, ISR_NAKED)
{
    PORTC |= (1 << PC4);
}

00000094 <__vector_1>:
  94:   sbi     0x08, 4 ; 8
```

- `ISR_NAKED` nu mai dezactivează întreruperile
- setarea unui bit pe 1 poate fi făcută cu o singură instrucțiune





```
#include <avr/interrupt.h>
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>

volatile int flag = 0;

ISR(INT0_vect)
{
    flag = 1;
}

int main()
{
    LCD_init();

    EIMSK |= (1 << INT0);

    sei();

    while (1)
    {
        if (flag)
        {
            LCD_print("PC4_pressed");
        }
    }
}
```

- Compilatorul nu poate crea dependențe de variabile între `main` și întreruperi
- `volatile` instruiește compilatorul să nu optimizeze în jurul acelei variabile
- Ce s-ar fi întâmplat dacă nu puneam `volatile`?

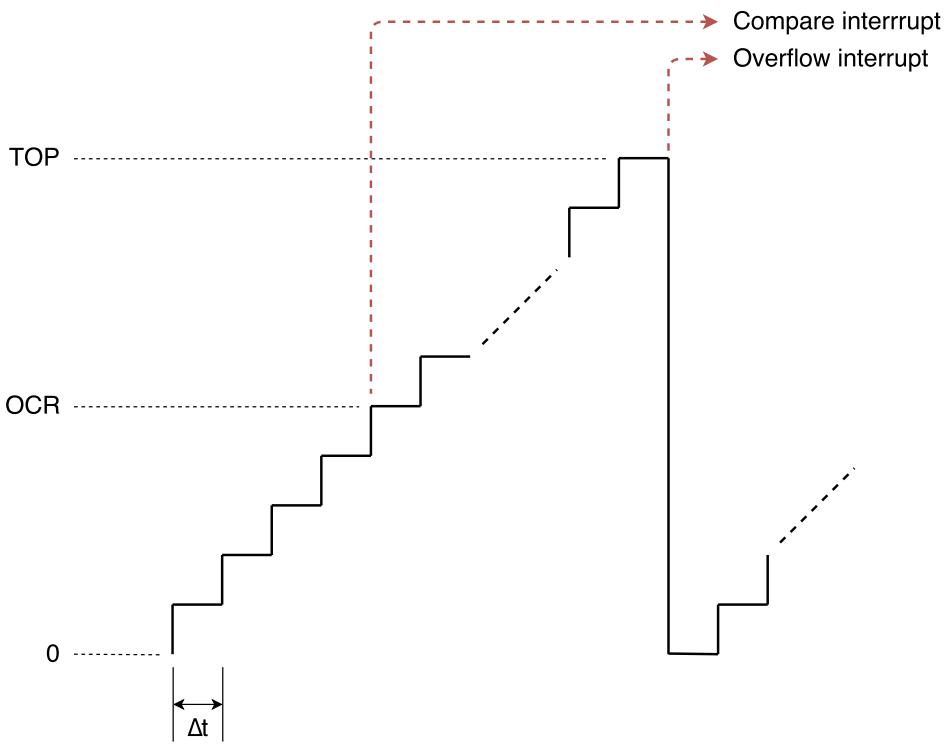


# Alte detalii de boot

```
0000007c <__ctors_end>:
  7c:   eor     r1, r1
  7e:   out     0x3f, r1        ; 63
  80:   ldi     r28, 0xFF       ; 255
  82:   ldi     r29, 0x08       ; 8
  84:   out     0x3e, r29       ; 62
  86:   out     0x3d, r28       ; 61
  88:   call    0x96    ; 0x96 <main>
  8c:   jmp     0xa0    ; 0xa0 <_exit>
```

- 0x3f este SREG
- 0x3e și 0x3d sunt SPH și SPL, stiva pe 16 biți
- de ce punem în stivă valoarea 0x08FF?

- Timerele sunt dispozitive care ne permit să
  - știm cât timp a trecut între două evenimente
  - numărăm evenimente externe
  - să executăm cod după un anumit timp sau la intervale regulate
- Timerele sunt implementate ca niște contoare
- Ca orice periferic I/O, vor avea registre I/O unde sunt configurate
  - Cum numără (până la cât, crescător/descrescător)
  - Cât de repede numără
  - Când sunt asociate acțiuni
  - Ce se execută în acele acțiuni




- TOP este valoarea până la care numără
- OCR este un prag la care se poate întâmpla o acțiune (întrerupere sau PWM)
- $\Delta t$ este dat de viteza cu care numără

$$\Delta t = \frac{PS}{f_{CPU}}$$

- Pe ATMega324 timerele apar sub forma Timer/Counter
- 0 și 2 sunt timere pe 8 biți, 1 este timer pe 16 biți
- TCCRxA și TCCRxB (x=0,1,2) - Control Register
  - Bitfield CS - Clock Select (cât de repede numără)
  - Bitfield WGM - Waveform Generation Mode (cum numără)
  - Bitfield COM - Compare Output (pentru PWM) Mode
- OCRxA, OCRxB - Output Compare Register (praguri)
- TCNT - Counter (contorul efectiv)
- TIMSK - interrupt mask register

- *No clock source* înseamnă că nu va număra
- Prescaler-ul are valori fixe
- Poate fi legat un semnal extern de ceas pe Timer0 și 1
- Pe timer 2 poate fi legat un alt cristal

**Table 16-6.** Clock Select bit description.

| CSn2 | CSn1 | CSn0 | Description |
|---|---|---|---|
| 0 | 0 | 0 | No clock source (Timer/Counter stopped). |
| 0 | 0 | 1 | $clk_{I/O}$/1 (No prescaling) |
| 0 | 1 | 0 | $clk_{I/O}$/8 (From prescaler) |
| 0 | 1 | 1 | $clk_{I/O}$/64 (From prescaler) |
| 1 | 0 | 0 | $clk_{I/O}$/256 (From prescaler) |
| 1 | 0 | 1 | $clk_{I/O}$/1024 (From prescaler) |
| 1 | 1 | 0 | External clock source on Tn pin. Clock on falling edge. |
| 1 | 1 | 1 | External clock source on Tn pin. Clock on rising edge. |

**Table 16-5.** Waveform Generation mode bit description (1).

| Mode | WGMn3 | WGMn2 (CTCn) | WGMn1 (PWMn1) | WGMn0 (PWMn0) | Timer/Counter mode of operation | TOP | Update of OCRnx at | TOVn flag set on |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | Normal | 0xFFFF | Immediate | MAX |
| 1 | 0 | 0 | 0 | 1 | PWM, Phase Correct, 8-bit | 0x00FF | TOP | BOTTOM |
| 2 | 0 | 0 | 1 | 0 | PWM, Phase Correct, 9-bit | 0x01FF | TOP | BOTTOM |
| 3 | 0 | 0 | 1 | 1 | PWM, Phase Correct, 10-bit | 0x03FF | TOP | BOTTOM |
| 4 | 0 | 1 | 0 | 0 | CTC | OCRnA | Immediate | MAX |
| 5 | 0 | 1 | 0 | 1 | Fast PWM, 8-bit | 0x00FF | BOTTOM | TOP |
| 6 | 0 | 1 | 1 | 0 | Fast PWM, 9-bit | 0x01FF | BOTTOM | TOP |
| 7 | 0 | 1 | 1 | 1 | Fast PWM, 10-bit | 0x03FF | BOTTOM | TOP |
| 8 | 1 | 0 | 0 | 0 | PWM, Phase and Frequency Correct | ICRn | BOTTOM | BOTTOM |
| 9 | 1 | 0 | 0 | 1 | PWM, Phase and Frequency Correct | OCRnA | BOTTOM | BOTTOM |
| 10 | 1 | 0 | 1 | 0 | PWM, Phase Correct | ICRn | TOP | BOTTOM |
| 11 | 1 | 0 | 1 | 1 | PWM, Phase Correct | OCRnA | TOP | BOTTOM |
| 12 | 1 | 1 | 0 | 0 | CTC | ICRn | Immediate | MAX |
| 13 | 1 | 1 | 0 | 1 | (Reserved) | – | – | – |
| 14 | 1 | 1 | 1 | 0 | Fast PWM | ICRn | BOTTOM | TOP |
| 15 | 1 | 1 | 1 | 1 | Fast PWM | OCRnA | BOTTOM | TOP |

Note: 1. The CTCn and PWMn1:0 bit definition names are obsolete. Use the WGMn2:0 definitions. However, the functionality and location of these bits are compatible with previous versions of the timer.

- Biții de COM activează semnalul de PWM pe canalul asociat (A sau B)
- Non-inverting generează un semnal cu duty cycle $D = \frac{1+prag}{1+TOP}$
- Inverting mode va avea $D = \frac{TOP-prag}{1+TOP}$
- Toggle mode va avea $D = 50\%$

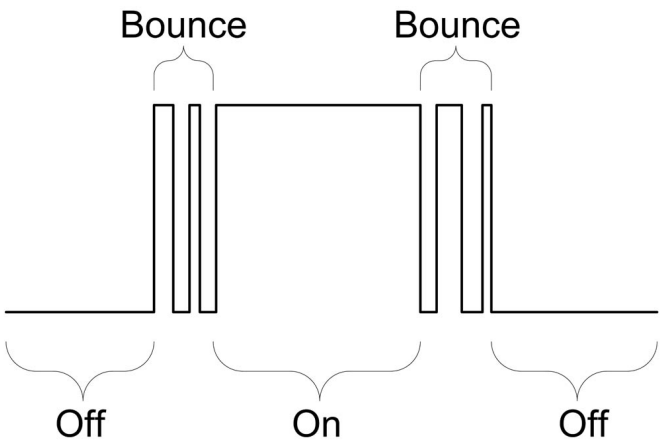
**Table 16-3.** Compare Output mode, fast PWM (1).

| COMnA1/COMnB1 | COMnA0/COMnB0 | Description |
|---|---|---|
| 0 | 0 | Normal port operation, OCnA/OCnB disconnected. |
| 0 | 1 | WGMn3:0 = 14 or 15: Toggle OC1A on Compare Match, OC1B disconnected (normal port operation). For all other WGM1 settings, normal port operation, OC1A/OC1B disconnected. |
| 1 | 0 | Clear OCnA/OCnB on Compare Match, set OCnA/OCnB at BOTTOM (non-inverting mode) |
| 1 | 1 | Set OCnA/OCnB on Compare Match, clear OCnA/OCnB at BOTTOM (inverting mode) |

- AVR este o arhitectură pe 8 biți, dar se dorește totuși accesul "atomic" la valori care depășesc 255
- Registrele pe 16 biți sunt împărțite într-un byte `high` și un byte `low`
- Pentru a face accesul atomic, perifericele intermediază accesul la aceste adrese cu un *latch*:
  - La citirea byte-ului `low`, se citește automat și byte-ul high și se stochează într-o locație temporară
  - La citirea byte-ului `high` valoarea citită este tot timpul cea din locația temporară
  - Scrierea se face în ordinea opusă (întâi byte-ul `high` și apoi `low`)
  - Motivația este dată de variația în timp, vrem byte-ul `low` să corespundă în timp ultimei citiri/scrieri
- Pentru C compilatorul poate face singur aceste operații în ordinea corectă

- Semnale pătratice periodice
  - Duty cycle
  - Frecvență
- De ce ne trebuie
  - Pot controla foarte multe dispozitive
  - Semnale PWM în tensiune pot controla drivere de LEDuri, drivere de motoare

- Utilizări
  - Modul default pentru toate timerele
  - Nu ne lasă să modificăm până la cât numără, deci poate fi folosit pentru a genera evenimente la o frecvență fixată de rezoluția timerului
    - Pentru un timer de 8 biți, frecvența la care va face overflow va fi $f_{OVF} = \frac{f_{CPU}}{PS \cdot 256}$
    - Controlul frecvenței este limitat la prescaler sau la evenimente externe




- Folosiri → aplicații care nu au nevoie de o anumită frecvență
- Exemplu: o variantă de debouncing pentru butoane
- Cum am putea face debouncing cu ajutorul unui timer?
  - Nu avem nevoie de o frecvență anume, trebuie doar să știm cât timp a trecut de la ultima apăsare




# Exemplu mod normal

```
volatile uint16_t   lastEvent = 0;
volatile uint8_t    buttonState = 0;  // 0 normal, 1 apasat

// 250 ms
#define DELAY   (16000000UL / 1024 / 4)

ISR(PCINT2_vect)
{
    if (buttonState != ((PINC & (1 << PC0)) == 0))
    { // daca butonul e apasat si noi credeam ca nu este
        if ((TCNT1 - lastEvent > DELAY))
        {
            buttonState = ((PINC & (1 << PC0)) == 0);
            buttonState? button_press() : button_release();
            lastEvent = TCNT1;
        }
    }
}

int main()
{
    TCCR1B = (5 << CS10); // prescaler 1024

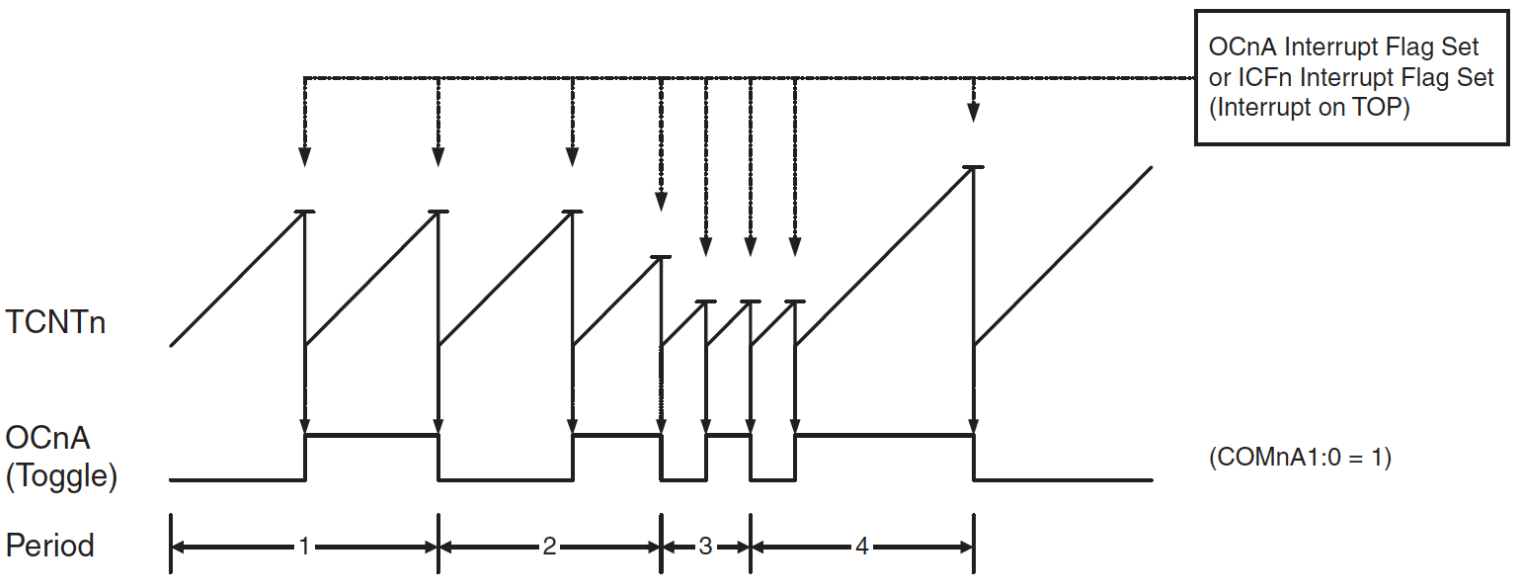
    DDRC    &= ~(1 << PC0);      // PC0 ca intrare
    PCICR   |= (1 << PCINT1);    // PC0 este PCINT16, deci e in al
    PCMSK2  |= (1 << PCMSK16);   // treilea registru PCMSK

    sei();

    while (1);
    return 0;
}
```

- Modul de bază în care contorul se resetează când atinge un prag (fie OCR, fie ICR)
- Putem controla frecvența cu care ajungem la *Compare match*
- Dar nu ajungem aproape niciodată la overflow
- $f_{CMP} = \frac{f_{CPU}}{PS \cdot (1 + OCRxA)}$
- Spre deosebire de alte moduri, modificările la prag se fac instantaneu
  - Ex: pragul este 100, contorul este 50, mutăm pragul la 25
  - La ce valoare se va reseta?
  - Pentru că update-ul se face imediat, nu la fiecare reset, contorul va ajunge la valoarea maximă (0xFF sau 0xFFFF)

**Figure 16-6.** CTC mode, timing diagram.



- Un ceas are nevoie să numere secundele
- Vrem să numărăm o secundă în hardware, iar *numărul de secunde* să îl numărăm în software
- cu prescalerul maxim de 1024, $f_{CMP_{MAX}} = \frac{f_{CPU}}{1024 \cdot 256} \simeq 61\,Hz$
- Nu putem număra o secundă cu timerele de 8 biți
- Cu un timer de 16 biți $\rightarrow OCRxA = \frac{f_{CPU}}{PS \cdot 1\,Hz} - 1 = 15624$

```
...

volatile uint32_t nrSeconds = 0;

ISR(TIMER1_COMPA_vect)
{
    nrSeconds++;
}

int main()
{
    OCR1A  = (F_CPU / 1024) - 1;          // 15624
    TCCR1B = (5 << CS10) | (1 << WGM12); // prescaler 1024 si mod CTC cu TOP la OCR1A

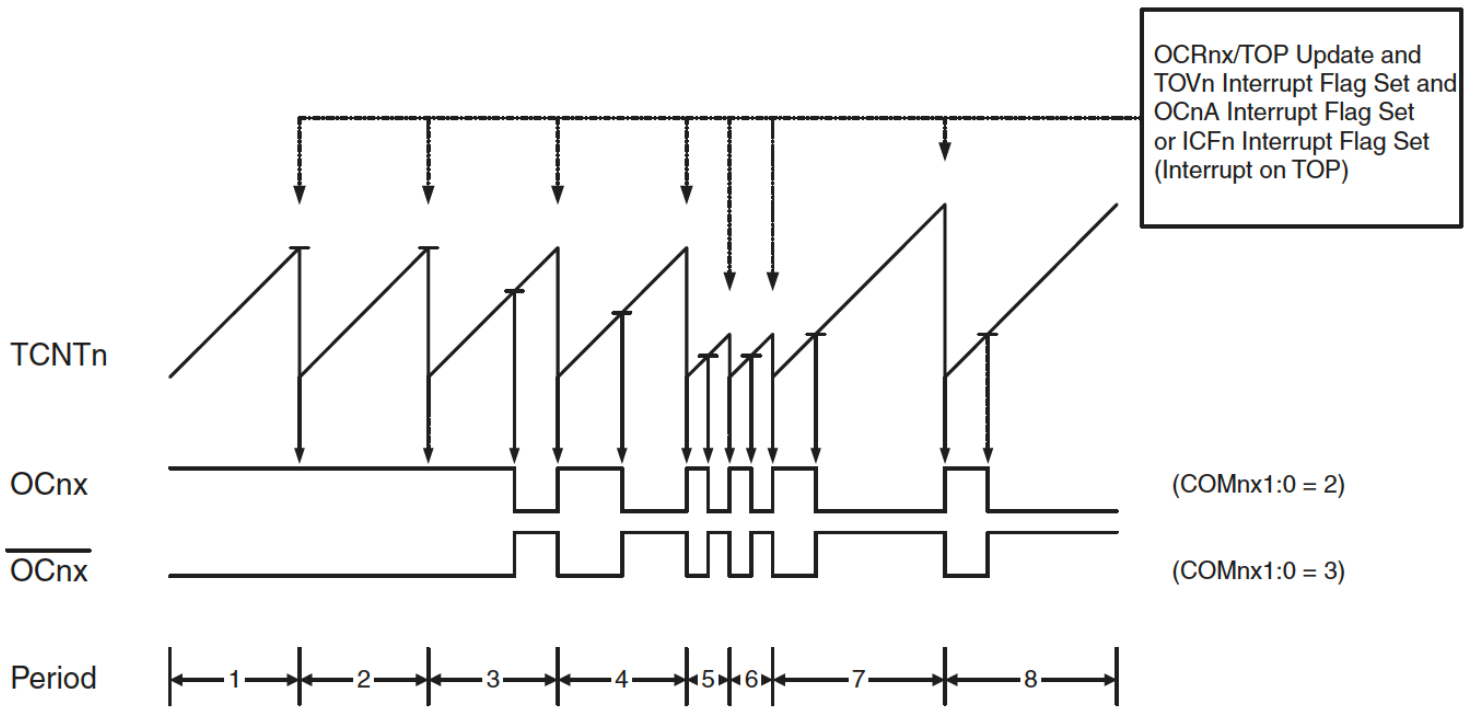
    TIMSK |= (1 << OCIE1A);               // intrerupere pe OCR1A

    sei();

    while (1);
    return 0;
}
```

- Moduri similare cu modul normal sau CTC, dar cu update de prag la reset
  - Acest mod oferă garanții pentru semnalele PWM generate
- Modul cel mai folosit de la laborator!

**Figure 16-7.** Fast PWM mode, timing diagram.



- Vom da ca exemplu folosirea unui singur timer pentru două scopuri
  - Ca în laborator, un canal de PWM este folosit pentru aprinderea unui LED
  - Alt canal de PWM este folosit pentru speaker
- Nevoi:
  - Canalul 1 (LED): Vrem să putem modifica ușor duty-cycle-ul, frecvența poate fi oricât atât timp cât este mai mare de 500Hz
  - Canalul 2 (Speaker): Vrem să putem modifica frecvența, duty cycle-ul trebuie să fie fixat la 50%

- 2 semnale $\rightarrow$ 2 canale PWM
- Pentru a modifica frecvența putem folosi doar `OCR1A` sau `ICR`
  - LEDul este pe canalul A, deci avem nevoie de `OCR1A` pentru acel canal (prag $\rightarrow$ duty cycle)
  - **trebuie** să folosim `ICR` ca prag (ICR este Input Capture Register, folosit pentru a reține momentul apariției unor evenimente externe)
- Concluzii:
  - Frecvența este setată de `ICR1`
  - pragul pentru canalul A este `OCR1A` și determină luminozitatea pe LED
  - duty cycle-ul pe canalul B trebuie să fie 50%, putem face asta în două moduri diferite

```
...

uint32_t nrSeconds = 0;

void set_speaker_frequency(int fr)
{
    uint32_t threshold = F_CPU / 2 / fr - 1; // inversul formulei de mai devreme

    threshold = MIN(threshold, 0xFFFF);
    threshold = MAX(threshold, 3);

    ICR = threshold;     // cast implicit la 16 biti
}

int main()
{
    // prescaler 1 si mod CTC cu TOP la ICR1
    TCCR1B = (1 << CS10) | (1 << WGM13) | (1 << WGM12);
    TCCR1A = (1 << WGM11);

    TCCR1A |= (2 << COM1A0);     // non-inverting mode pe canalul A
    TCCR1A |= (3 << COM1B0);     // toggle mode pe canalul B

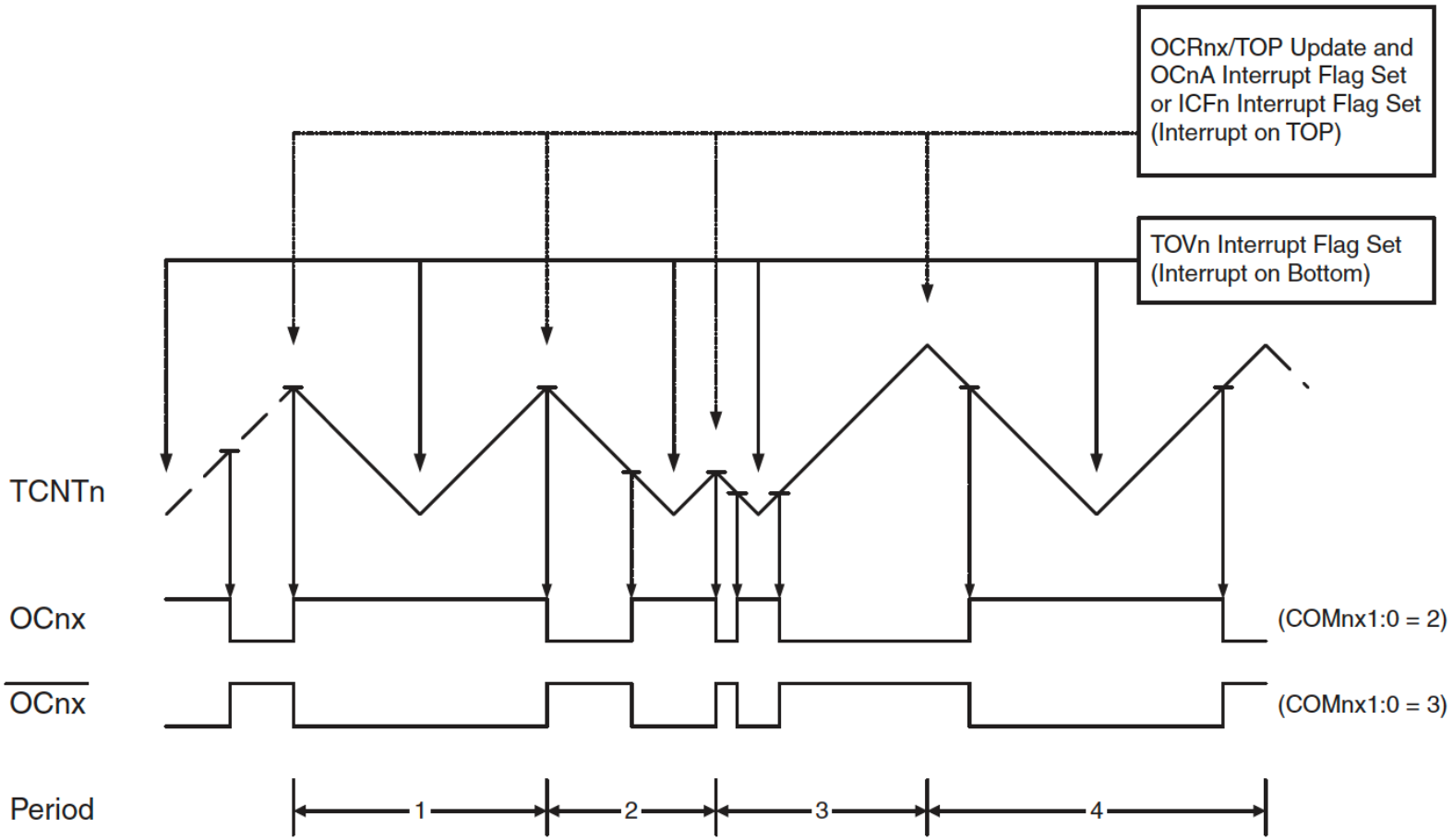
    DDRD |= (1 << PD4) | (1 << PD5); // canalele de PWM trebuie setate ca iesiri

    while (1);
    return 0;
}
```

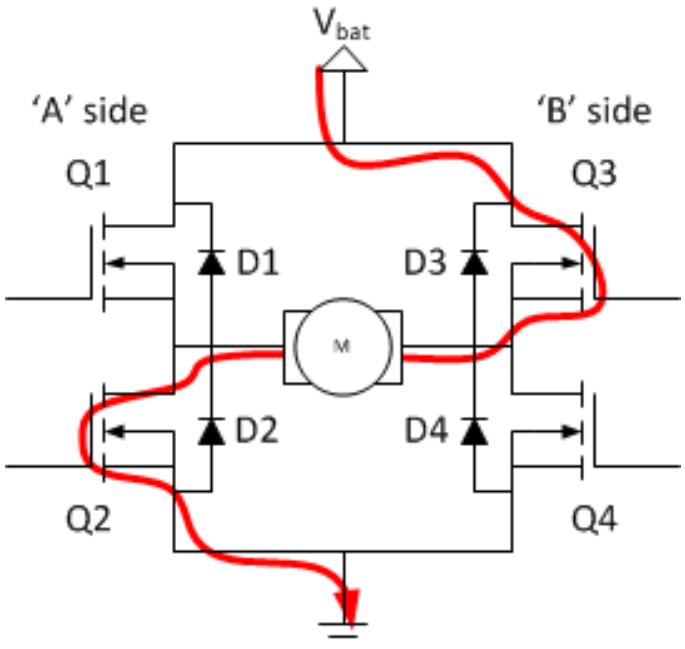
- Care este gama de frecvențe posibilă pe speaker-ul din exemplul precedent?
  - Gama de praguri valide este $[3, 65535]$
  - $f_{Speaker} = \frac{f_{CPU}}{2 \cdot 1 \cdot (prag+1)}$
  - Gama de frecvențe este atunci $[122Hz, 2MHz]$
  - Cum putem schimba această gamă?
  - Dacă am fi folosit prescaler-ul de 1024, gama era $[0.12Hz, 1.95kHz]$
- Trebuie să folosim un prescaler care ne permite să avem întreaga gamă de frecvențe audibile $[20Hz, 20kHz]$, dacă este posibil

- Folosit tot pentru generarea semnalelor PWM
- Acolo unde tranzițiile semnalului sunt în general nedorite
- Contorul numără între 0 și TOP, apoi înapoi la 0
  - Dacă facem tranziția pe semnalul PWM la prag, obținem un semnal cu același duty cycle ca fastPWM, dar cu frecvență de 2 ori mai mică
  - *Clear when down-counting, set on up-counting*
  - *Set when down-counting, clear on up-counting*

**Figure 16-8.** Phase Correct PWM mode, timing diagram.

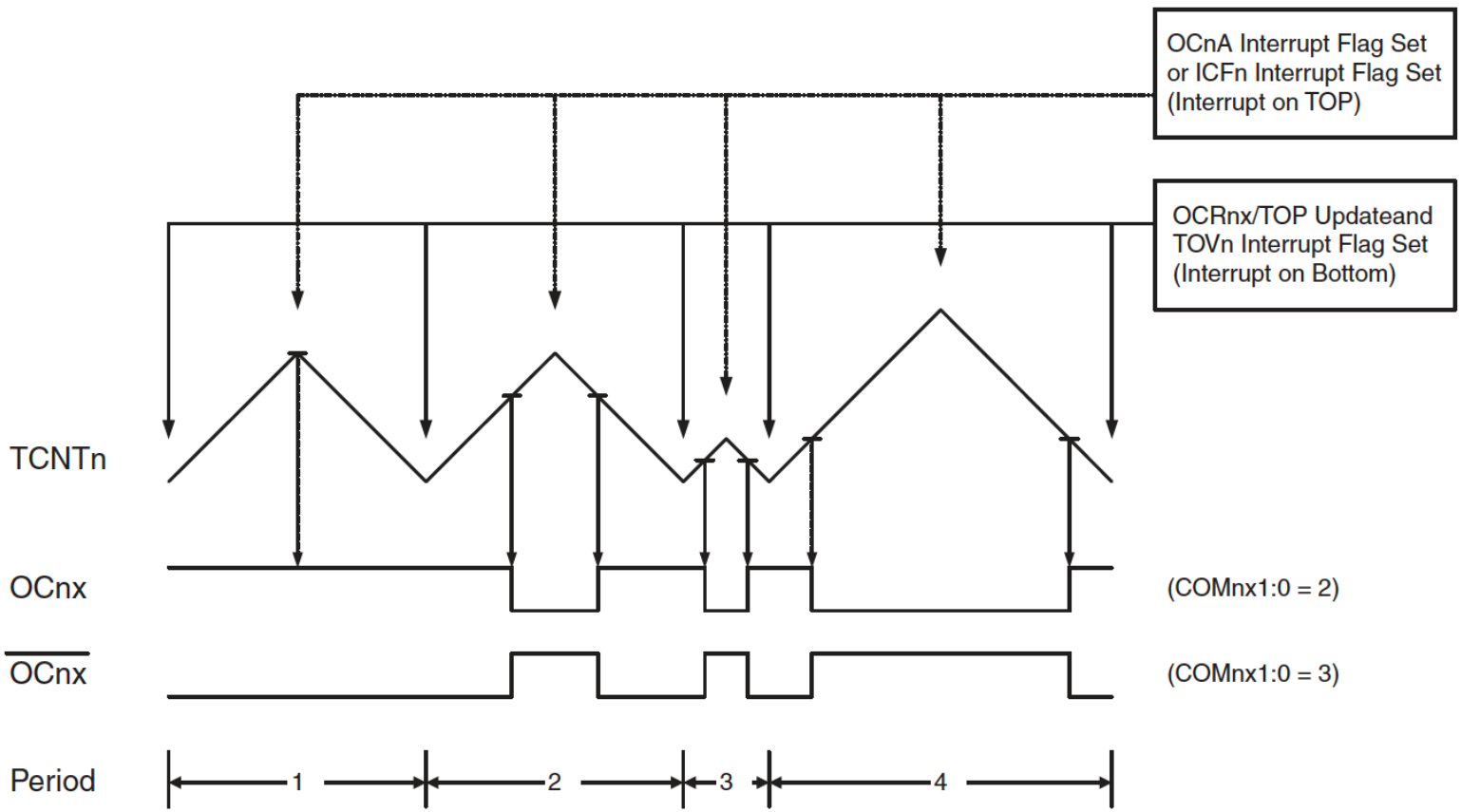


- Un motor nu poate fi controlat direct din pini
- Dacă folosim un singur tranzistor (*driver*), putem porni motorul într-un singur sens
- Pentru a porni motorul DC în ambele sensuri este nevoie de o punte H

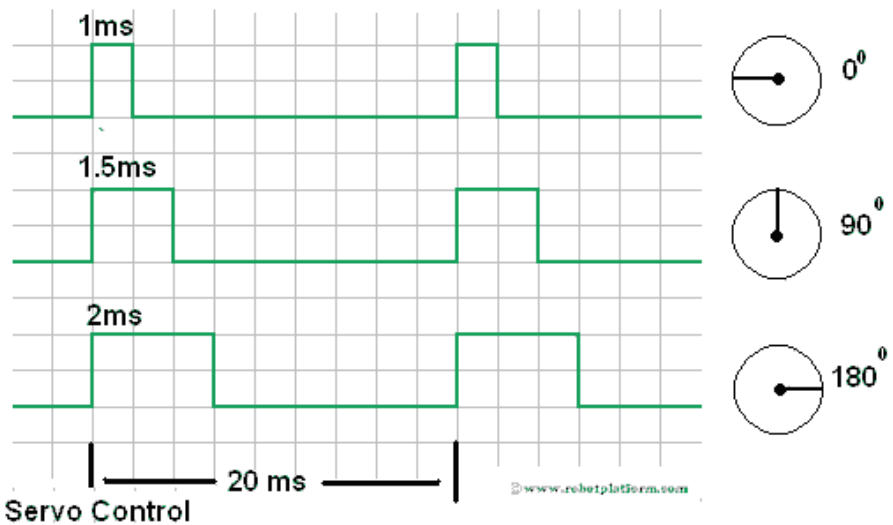


- La fel ca modul PWM phase correct, dar update-ul pe prag se întâmplă când se ajunge la 0 $\rightarrow$ pulsurile generate sunt întotdeauna consistente

**Figure 16-9.** Phase and Frequency Correct PWM mode, timing diagram.








- Motoare cu 3 pini: VCC, GND, semnal control
- Semnalul de control este tipic la 5V și cu pulsuri de 1-2ms la intervale regulate care îi dau motorului *comanda*.
- Motorul încearcă să îndeplinească comenzile în timp util

```
...

void init_servo()
{
    // FastPWM cu TOP la OCR0A, prescaler de 1024
    TCCR0A = (1 << WGM01);
    TCCR0B = (5 << CS00) | (1 << WGM03) | (1 << WGM02);

    OCR0A = (F_CPU / 1024 / 50) - 1;     // 50Hz <==> 20ms perioada
}

// primeste un unghi intre 0 si 180
void rotate_servo(int angle) // in degrees
{
    angle = MAX(angle, 0);
    angle = MIN(angle, 180);

    OCR0B = OCR0A / 20;                                  // 1ms
    uint16_t fraction_times_100 = angle * 10 / 18; // intre 0 si 1 x100
    OCR0B += OCR0A * fraction_times_100 / 2000;

}

...
```

- Putem forma un timer de 24 (16 + 8) de biți pe ATMega324 înlănțuind Timer0 cu Timer1
  - Unul dintre timere poate genera un semnal PWM când face overflow
  - Semnal poate fi "input" pentru al doilea timer - Bitfieldul de Clock Select poate folosi un pin extern ca ceas pentru counter în loc de ceasul sistemului
  - Semnalul de PWM de la unul dintre timere poate fi considerat semnal de ceas pentru următorul




# Ce este un watchdog?

- Watchdog-ul este un timer special al sistemului
- Când timer-ul ajunge la o anumită valoare timer-ul "expiră"
  - Controller-ul este fie resetat fie intră într-o întrerupere de watchdog
- Pentru a preveni asta, programul de pe microcontroller trebuie să reseteze contorul periodic *petting the watchdog*
- Astfel se garantează că programul nu intră într-o stare de deadlock sau o altă stare invalidă




# Watchdog - exemplu cod

```
...

ISR(WDT_vect)
{
    do_stuff();                                  // retine informatii relevante despre
                                                 // starea anormala in care se gaseste
}

int main()
{
    WDTCSR = (1 << WDTON) | (1 << WDIE);  // watchdog enable, cu intrerupere
    WDTCSR |= (9 << WDP0);                       // timeout de 8s

    sei();

    while (1)
    {
        wdt_reset();                             // pet/reset the watchdog
        do_normal_stuff();                       // bucla normala
        _delay_ms(250);
    }
    return 0;
}
```